

TIME-GUARDED AUTOMATA

Definition 1. A Time-Guarded Nondeterministic Word Automaton (TGW) is an expansion of the usual Nondeterministic Word Automaton.

Intuitively, a TGW can be constructed by taking an NFA, and setting certain transitions to be “closed” or “inactive” at certain “times”.

Formally, a TGW A would be defined similarly to an NFA, by a quintuplet: $A = (Q, \Sigma, \delta, q_0, F)$, where:

Q - A finite set of states

Σ - A finite set of symbols, AKA the Alphabet

δ - The transition function:

The transition function differs from the one of the usual NFA, because in addition to taking the current state and symbol as arguments, it also takes t which is the number of steps taken so far:

$$\delta : Q \times \Sigma \times \mathbb{N} \rightarrow 2^Q$$

q_0 - An initial state

F - A subset of accepting states

Example 2. Let us take a look on a TGW A over an alphabet $\Sigma = \{a, b\}$, first as a diagram:

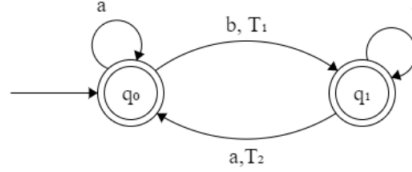


FIGURE 0.1. A Time-Guarded Automaton

To complete the definition from the diagram, one could define T_1 and T_2 as subsets of $\mathbb{N}$:

$$T_1 = \{t : t \in \mathbb{N} \wedge t = 1 \pmod{2}\}$$

$$T_2 = \{t : t \in \mathbb{N} \wedge t = 0 \pmod{3}\}$$

(From now on $t \in \mathbb{N}$ will be always implied unless stated otherwise)

Note - The transitions in the diagram without a “ T_n ” are always active.

One could also describe the same automaton textually:

$$Q = \{q_0, q_1\}, F = \{q_0, q_1\}$$

$$\delta(q_0, a, t) = \{q_0\}, \delta(q_0, b, t) = \begin{cases} \{q_1\} & t \in T_1 \\ \emptyset & \text{otherwise} \end{cases}$$

$$\delta(q_1, b, t) = \{q_1\}, \delta(q_1, a, t) = \begin{cases} \{q_0\} & t \in T_2 \\ \emptyset & \text{otherwise} \end{cases}$$

Another way to describe δ would be to view it as the set of all transitions that exist in the automaton.

A transition would be written as $\langle p, \sigma, T, q \rangle$ and describe a transition from state p

to q , with a symbol σ which is active on a guard $T \subseteq \mathbb{N}$. For the set-representation of δ we will use the capital Δ . In our example:

$$\Delta = \{\langle q_0, a, \mathbb{N}, q_0 \rangle, \langle q_0, b, T_1, q_1 \rangle, \langle q_1, a, T_2, q_0 \rangle, \langle q_1, b, \mathbb{N}, q_1 \rangle\}$$

Let us take a look on a few words and see if they are in $L(A)$. Similarly to *NFA*, a word w is accepted by a TGW A if there exists an accepting run of A on w .

$ab \in L(A)$: Because the run $q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_1$ is possible. This is because after taking the step $q_0 \xrightarrow{a} q_0$, the number of steps t would be 1, which is in T_1 , and thus the transition $q_0 \xrightarrow{b} q_1$ would be active, leaving us on the accepting state q_1 at the end of this run.

$ba \notin L(A)$: At the start of the run, the time $t = 0$. $0 \notin T_1$, there for the transition $q_0 \xrightarrow{b} q_1$ is inactive, and because it is the only transition from q_0 with b , the word is rejected.

Regular Guards. In order to limit the space of possible “guards” for example, to exclude a guard such as:

$$T = \{t : t \text{ is prime}\}$$

we could demand a guard $T \subseteq \mathbb{N}$ to be a regular language over a singleton alphabet, i.e demand the existence of an automaton over a singleton alphabet $\{1\}$ which language is $\{1^t : t \in T\}$. Note that the guards in the example above can be viewed as regular languages.

Succinctness. When we demand regular guards, we can come up with a way to translate a TGW to a normal NFA, using the automata that recognize the guards. It could be intuitively viewed by running simultaneously on the original TGW automaton, and all the guards’ automata, and on every “timed” transition, checking if the respective “guard automaton” is on an accepting state.

For convinience we will look at δ in its transition-set representation Δ . Now, given a TGW A with guards $T_1, \dots, T_k$, and their respective NFA’s $A_{T_1}, \dots, A_{T_k}$ we will construct the NFA A' such that:

$$L(A') = L(A)$$

Notations :

- $A = (Q, \Sigma, \Delta, q_0, F)$
- $A' = (Q', \Sigma, \Delta', q'_0, F')$
- $A_{T_i} = (Q^i, \{1\}, \Delta^i, q_0^i, F^i)$

Construction:

- $Q' = Q \times Q^1 \times \dots \times Q^k$, and $q'_0 = (q_0, q_0^1, \dots, q_0^k)$
- A transition from two states in Q' , $p' = (p, p^1, \dots, p^k)$ to $q' = (q, q^1, \dots, q^k)$, with a letter σ would be included in Δ' if all conditions hold:
 - There is $1 \leq i \leq k$ such that $\langle p, \sigma, T_i, q \rangle \in \Delta$ and $p^i \in F^i$. i.e - in the original automaton A we have a transition from p to q with a guard T_i , and the guard-automaton is on an accepting state.
 - For all $1 \leq j \leq k$ we have that $\langle p^j, q^j, 1 \rangle \in \delta^j$. i.e the respective transition exists in all guard automata

- $F' = \{(q, q^1, \dots, q^k) : q \in F\}$

Note - from now on we will look mostly on automata over infinite words, such as Büchi automata.

Stability.

Definition 3. k -Latency

Given a *TDBW* (Time-Guarded Büchi automaton over words) A , and $k \in \mathbb{N}$, the k -late language of A , $L_k(A)$ is the set of words $\in \Sigma^*$ that are accepted by A if the time t starts at k instead of 0.

Remark. If *TDBW* A is given in the form of a main automaton and its guard automata, we can modify the guard automata in order to achieve the *TDBW* A_k , which language is $L_k(A)$. This could be done by setting the Q_0 of each guard automaton to the set of states which are achievable with k steps from its original Q_0 .

Definition 4. k -Stability

Given a *TDBW* A , an (infinite) word $w \in \Sigma^\omega$ is k -stable over A if both $w \in L(A)$ and $w \in L_k(A)$.

A *TDBW* A is said to be k -stable if all the words in $L(A)$ are k -stable, i.e. $L(A) \subseteq L_k(A)$.

History Guarded Automata Emptiness is PSPACE Complete

Definition 5. A **History Guarded (HG) Automaton** is an automaton $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$ with guard automata $\mathcal{A}_1, \dots, \mathcal{A}_n$. It functions the same as a “regular” automaton but its transitions can be guarded by one (or more) of the automata $\mathcal{A}_1, \dots, \mathcal{A}_n$. Upon reaching a guarded transition, the transition can be taken only if the prefix of the word read to this point is accepted by the guard automaton (or automata, if it’s a guard set). The word prefix that was read up to a certain point can also be called the current “history state”. If $\mathcal{A}$ has more symbols to read but no active transitions, it transitions to a rejecting sink.

Definition 6. A language $L \subseteq \Sigma^*$ is **co-safe** if every word $w \in L$ has a prefix $x \in \Sigma^*$ such that for every $z \in \Sigma^*$, we have that $xz \in L$.

Definition 7. A language $L \subseteq (\Sigma \cup \{\$ \})^*$ is **hinted co-safe (HCS)** if every $w \in L$ is of the form, $x\$y$ for $x, y \in \Sigma^*$, and for every $z \in \Sigma^*$, we have that $x\$z \in L$. An automaton whose language is HCS is called a “HCS automaton”.

Theorem 8. *The following problem is PSPACE-Hard: Given n HCS DFAs $\mathcal{A}_1, \dots, \mathcal{A}_n$, decide whether $\bigcap_{i=1}^n L(\mathcal{A}_i) = \emptyset$.*

Proof. We show a reduction from the following PSPACE complete problem: “Given a Turing machine T of space complexity $s(n)$, decide whether the empty tape is accepted by T .”

The reduction returns $2 \cdot s(0) + 1$ HCS DFAs: $S = \{\mathcal{A}_1, \dots, \mathcal{A}_{s(0)}, \mathcal{A}'_1, \dots, \mathcal{A}'_{s(0)}, \mathcal{A}_\#\}$. The empty tape is accepted by T iff there exists a word that is accepted by all automata. The automata run on an encoding of a sequence of configurations, and their “goal” is to check whether it’s a valid and accepting computation of T on the empty tape. $\square$

Let $T = \langle \Gamma, Q, \rightarrow, q_0, F \rangle$, where Γ is the alphabet, Q is the set of states, $\rightarrow \subseteq Q \times \Gamma \times Q \times \Gamma \times \{L, R\}$ is the transition relation (we use $(q, a) \rightarrow (q', b, \Delta)$ to indicate that when T is in state q and it reads the input a in the current tape cell, it moves to state q' , writes b in the current tape cell, and its reading head moves one cell to the left/right, according to Δ), q_0 is the initial state, and $F \subseteq Q$ is the set of accepting states.

?proofname? We encode a configuration of T by a string $\# \gamma_1 \gamma_2 \dots (q, \gamma_i) \dots \gamma_{s(n)}$. That is, a configuration starts with $\#$, and all its other letters are in Γ , except for one letter in $Q \times \Gamma$. The meaning of such a configuration is that the j 'th cell in T , for $1 \leq j \leq s(n)$, is labeled γ_j , the reading head points at cell i , and T is in state q . For example, the initial configuration of T is $\#(q_0, b)b \dots b$ (with $s(n) - 1$ occurrences of b 's) where b stands for an empty cell. We can now encode a computation of T by a sequence of configurations.

Let $\Sigma = \{\#\} \cup \Gamma \cup (Q \times \Gamma)$ and let $\# \sigma_1 \dots \sigma_{s(n)} \# \sigma'_1 \dots \sigma'_{s(n)}$ be two successive configurations of T . We also set σ_0, σ'_0 , and $\sigma_{s(n)+1}$ to $\#$. For each triple $\langle \sigma_{i-1}, \sigma_i, \sigma_{i+1} \rangle$ with $1 \leq i \leq s(n)$, we know, by the transition relation of T , what σ'_i should be. In addition, the letter $\#$ should repeat exactly every $s(n) + 1$ letters. Let $next(\langle \sigma_{i-1}, \sigma_i, \sigma_{i+1} \rangle)$ denote our expectation for σ'_i . That is,

- $next(\langle \gamma_{i-1}, \gamma_i, \gamma_{i+1} \rangle) = next(\langle \#, \gamma_i, \gamma_{i+1} \rangle) = next(\langle \gamma_{i-1}, \gamma_i, \# \rangle) = \gamma_i$.
- $next(\langle (q, \gamma_{i-1}), \gamma_i, \gamma_{i+1} \rangle) = next(\langle (q, \gamma_{i-1}), \gamma_i, \# \rangle) =$

$$\begin{cases} \gamma_i & \text{If } (q, \gamma_{i-1}) \rightarrow (q', \gamma'_{i-1}, L) \\ (q', \gamma_i) & \text{If } (q, \gamma_{i-1}) \rightarrow (q', \gamma'_{i-1}, R) \end{cases}$$
- $next(\langle \gamma_{i-1}, (q, \gamma_i), \gamma_{i+1} \rangle) = next(\langle \#, (q, \gamma_i), \gamma_{i+1} \rangle) =$
 $next(\langle \gamma_{i-1}, (q, \gamma_i), \# \rangle) = \gamma'_i$ where $(q, \gamma_i) \rightarrow (q', \gamma'_i, \Delta)$.
- $next(\langle \gamma_{i-1}, \gamma_i, (q, \gamma_{i+1}) \rangle) = next(\langle \#, \gamma_i, (q, \gamma_{i+1}) \rangle) =$

$$\begin{cases} \gamma_i & \text{If } (q, \gamma_{i+1}) \rightarrow (q', \gamma'_{i+1}, R) \\ (q', \gamma_i) & \text{If } (q, \gamma_{i+1}) \rightarrow (q', \gamma'_{i+1}, L) \end{cases}$$
- $next(\langle \sigma_{s(n)}, \#, \sigma'_1 \rangle) = \#$.

Consistency with $next$ now gives us a necessary condition for a trace to encode a legal computation. In addition, the computation should start with the initial configuration.

Finally, we require a configuration in which the machine either accepts or rejects to be followed by a $\$$ rather than $\#$.

We now describe the structure of the automata. Say the configurations in a computation are numbered starting at 0. The goal of the automata $\mathcal{A}_i$ is to remember the i 'th symbol (and its immediate neighbors) in each even numbered encoding of a configuration, and verify that the i 'th cell in the next configuration encoding matches the $next$ function. The automata $\mathcal{A}'_i$ do the same but for odd numbered configurations.

The automata $\mathcal{A}_i$ all use the alphabet $\Sigma \cup \{\$\}$ and are structured with the following transitions, in this order:

- From q_0 there are i transitions with all $\sigma \in \Sigma$, to reach the $(i - 1)$ 'th letter in an encoding of a configuration
- Then, there are $|\Sigma|$ splits on each transition in $\{i - 1, i, i + 1\}$, afterward putting the automaton in one of $|\Sigma|$ paths to expect the symbol: $next(\langle \sigma_{i-1}, \sigma_i, \sigma_{i+1} \rangle)$ in the next configuration.

- Then $s(0) - i - 1$ more Σ transitions to reach the beginning ($\#$) of the next configuration
- A transition that:
 - On $\$$ goes to an accepting sink
 - On $\#$ continues on the path to expect $next(\langle \sigma_{i-1}, \sigma_i, \sigma_{i+1} \rangle)$ in the i 'th cell
- $i - 1$ more transitions to reach the i 'th cell of the next (odd numbered) configuration
- A transition to a rejecting sink if $\sigma'_i \neq next(\langle \sigma_{i-1}, \sigma_i, \sigma_{i+1} \rangle)$
- $s(0) - i$ more transitions to reach the beginning of the next (even numbered) configuration, and goes back to q_0

The automata $\mathcal{A}'_i$ are structured identically, but their initial state is the one on the path expecting b (the blank symbol) in the next configuration, except for $\mathcal{A}'_0$ which expects (q_0, b) , i.e the reading head on the first (blank) cell.

The DFA $\mathcal{A}_\#$ starts with validating the first cell is $\#$, and then has $s(0) + 1$ Σ transitions to verify that every $s(0) + 1$ cell has $\#$. If at certain point it encounters a symbol of the form (q_{acc}, σ) for some $\sigma \in \Gamma$, it moves to path which expects $\$$ on the start of the next configuration encoding, instead of $\#$. After seeing that $\$$ in the right spot, it moves to an accepting sink.

Assume first that the empty tape is accepted by T . Let c be an encoding of an accepting computation of T on the empty tape. We show that c is accepted by all the automata in S :

- $\mathcal{A}_i$ - As c is a valid calculation of the empty tape, in particular each i 'th cell in an odd numbered configuration is consistent with the output of the $next$ function on the $\langle i - 1, i, i + 1 \rangle$ cells from the previous configuration. In addition, a legal accepting calculation encoding should end with $\$$ in the last configuration, putting all $\mathcal{A}_i$ in an accepting sink.
- $\mathcal{A}'_i$ - By the same reasoning, the $\mathcal{A}'_i$ automata should approve the consistency of the even numbered configuration with the ones before them, and be put in an accepting sink by the $\$$ symbol at the end.
- $\mathcal{A}_\#$ - As c is a valid encoding of a calculation, it should include a $\#$ in the start of each configuration, as required by $\mathcal{A}_\#$. In addition, as c is an accepting calculation, the second to last configuration should have a cell of the form $\langle q_{acc}, \sigma_i \rangle$, and the next (also last) configuration should start with $\$$. This, by the structure of $\mathcal{A}_\#$, puts $\mathcal{A}_\#$ in an accepting sink.

Thus, c is accepted by all the DFAs $\mathcal{A} \in S$, and therefore $\bigcap_{\mathcal{A} \in S} L(\mathcal{A}) \neq \emptyset$.

For the other direction, we assume that there exists a word c such that for all $\mathcal{A} \in S$, we have $c \in L(\mathcal{A})$. We show that c is an encoding of an accepting computation of T on the empty tape. It holds that:

- $c \in L(\mathcal{A}_\#)$ and therefore it is a valid encoding of configurations that start with $\#$, with T being on an accepting state at the end.
- For all $i \in [1, s(0)]$ we have $c \in L(\mathcal{A}_i)$ and $c \in L(\mathcal{A}'_i)$, and therefore each configuration is consistent with its predecessor, according to $next$.

Therefore we have that c is a valid encoding of an accepting calculation of T on the empty tape, meaning it is indeed accepted by T .

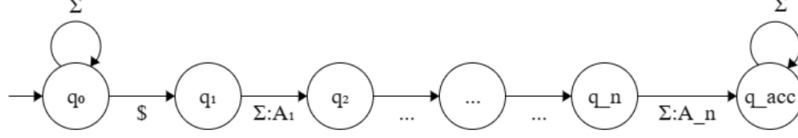


FIGURE 0.2. The returned automaton $\mathcal{A}$

Complexity of the reduction: The reduction calculates the structure of $O(s(0))$ automata, and each of them is of size $O(|\Sigma|^3)$ (maximum number of splits when reading the cells $\langle i-1, -, i+1 \rangle$) which puts it in PTIME as a function of $s(0)$ and $|\Sigma|$. $\square$

Theorem 9. *The following problem is PSPACE-Hard: Given a Deterministic Finite History-Guarded Automaton (DFHG) $\mathcal{A}$, with n DFAs as guards, decide whether $L(\mathcal{A}) = \emptyset$.*

Proof. We will prove it by showing a reduction from the PSPACE-Hard “HCS-Intersection” problem:

Given n DFAs $\mathcal{A}_1, \dots, \mathcal{A}_n$ over alphabet $\Sigma \cup \{\$, \}$, which languages are HCS, the reduction returns a DFHG $\mathcal{A}$, with the original $\mathcal{A}_1, \dots, \mathcal{A}_n$ as guards. $\mathcal{A}$ is defined as follows:

- $\mathcal{A} = \langle Q, \Sigma \cup \{\$, \}, \delta, q_0, F \rangle$, $Q = \{q_0, \dots, q_n, q_{acc}\}$, $F = \{q_{acc}\}$
- δ :
 - $\delta(q_0, \sigma) = \begin{cases} q_0 & \sigma \in \Sigma \\ q_1 & \sigma = \$ \end{cases}$
 - For $i \in [1, n-1]$: $\delta(q_i, \sigma) = \langle q_{i+1} : \mathcal{A}_i \rangle (\langle q_{i+1} : \mathcal{A}_i \rangle$ meaning a transition to q_{i+1} with $\mathcal{A}_i$ as guard)
 - $\delta(q_n, \sigma) = \langle q_{acc} : \mathcal{A}_n \rangle$
 - $\delta(q_{acc}, \sigma) = q_{acc}$

Correctness: Assume first that $\bigcap_{1 \leq i \leq n} L(\mathcal{A}_i) \neq \emptyset$. Thus, there is a word $w \in (\Sigma \cup \{\$, \})^*$ such that $w \in L(\mathcal{A}_i)$, for all $1 \leq i \leq n$. Since all the automata $\mathcal{A}_1, \dots, \mathcal{A}_n$ are HCS, the word w is of the form $x \cdot \$ \cdot y$, for $x, y \in \Sigma^*$. Also, for all $1 \leq i \leq n$, the word $x \cdot \$$ is a good prefix for all languages $L(\mathcal{A}_i)$, in the sense that for all words $z \in \Sigma^*$, we have that $x \cdot \$ \cdot z$ is in $L(\mathcal{A}_i)$. Let $t = \sigma_1 \cdot \sigma_2 \cdots \sigma_n$ be some word in Σ^n . We claim that the word $x \cdot \$ \cdot t$ is in $L(\mathcal{A})$. Indeed, since $x \cdot \$$ is a good prefix for all the languages $L(\mathcal{A}_i)$, then for all $1 \leq i \leq n$, we have that $x \cdot \$ \cdot \sigma_1 \cdot \sigma_2 \cdots \sigma_i$ is in $L(\mathcal{A}_i)$. Accordingly, a run of $\mathcal{A}$ that loops in q_0 while reading w , moves to q_1 with $\$$, and then, for $1 \leq i < n$, moves with σ_i from q_i to q_{i+1} , and finally moves with σ_n to q_{acc} , is a legal and accepting run of $\mathcal{A}$ on $x \cdot \$ \cdot t$. Thus, $L(\mathcal{A}) \neq \emptyset$ and we are done.

For the other direction, assume that $L(\mathcal{A}) \neq \emptyset$. Consider a word $w \in L(\mathcal{A})$. By the structure of $\mathcal{A}$, the word w must be of the form $x \cdot \$ \cdot y$, for $x, y \in \Sigma^*$, and $|y| \geq n$. Let $\sigma_1 \cdot \sigma_2 \cdots \sigma_n$ be the prefix of y of length n . Moreover, the guards on the transitions of $\mathcal{A}$ guarantees that for all $1 \leq i \leq n$, we have that the word

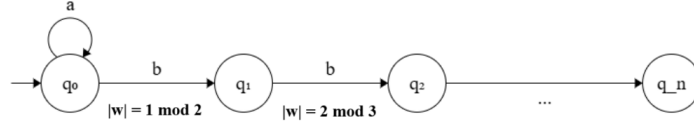
$x \cdot \$ \cdot \sigma_1 \cdot \sigma_2 \cdots \sigma_i$ is in $L(\mathcal{A}_i)$. Since all the automata $\mathcal{A}_1, \dots, \mathcal{A}_n$ are HCS, it follows that $x \cdot \$$ is a good prefix for $L(\mathcal{A}_i)$, implying that $w \in L(\mathcal{A}_i)$ for all $1 \leq i \leq n$. Thus, $\bigcap_{1 \leq i \leq n} L(\mathcal{A}_i) \neq \emptyset$, and we are done.

Complexity of the reduction: The reduction returns the original n automata with the additional automaton $\mathcal{A}$ which size is $O(n)$, therefore the reduction is in PTIME. $\square$

Claim 10. In the worst case, the translation from HG DFA to DFA results in an exponential blow-up in the number of states.

Proof. Let us look at an example of a language family for which the blow-up in the number of states from HG DFA to normal DFA is exponential:

Let A_n be the following family of HG DFA:



Such that for $i \in \{1, \dots, n\}$, the guard on the transition that leads to q_i is $|w| = i \bmod p_i$, where p_i is the i 'th prime. We can see that the i 'th guard only accepts words of the form $a^x b^i$ where x is a multiple of p_i . Therefore, from the structure of the automaton it follows that only words of the form $a^x b^n$ are accepted, where x is a multiple of the first n primes. For example, for $n = 3$, A_3 will accept only words of the form $a^x b^3$ where x is a multiple of the primes 2, 3, 5, i.e a multiple of 30.

Therefore, for the length of the shortest word accepted by A_n , notated $l_{min}(A_n)$ we get:

$$l_{min}(A_n) > \prod_{i=1}^n p_i \geq 2^n$$

That is, exponential in n .

In order to calculate the number of states of A_n , we'll first have to consider the number of states in its guards. For two integers $j < k$, in order for a guard (A simple DFA) to accept only words of length $j \bmod k$, it must be able to "count to k ", that is, have at least k states. Therefore the total number of states for the guards must be greater than $\sum_{i=1}^n p_i$. Asymptotically speaking, according to the prime number theorem, this sum tends to $\sum_{i=1}^n i \cdot \log i$, which is bounded by $n^2 \log n$. After adding the $n + 1$ states of the main automaton A_n , we get that the total number of states is still asymptotically bounded by $n^2 \log n$, that is, polynomial in n .

In conclusion, we get that for this automata family, the shortest word accepted by each automaton is exponential by the automaton's number of states.

In order to identify such language $L(A_n)$, a "normal" DFA would need to have a number of states that's proportional to $\prod p_i$, that is, exponential in n .

Finally, we get that for this language family, the blow-up in the number of states from HG DFA to normal DFA is exponential. $\square$

Transition Odds and Lucky Words

For the purposes of this part we will only look at **Time-Guarded-Automata with modular guards**, i.e. " $|w| = x \bmod y$ ".

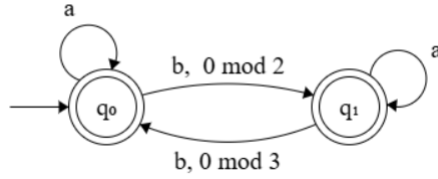
Definition 11. The **cycle length** of a time-guard " $|w| = x \bmod y$ " is y .

Definition 12. The LCM of a time-guarded automaton $A = \langle Q, \Sigma, \delta, q_0, F, \{G_1, \dots, G_n\} \rangle$ is the least-common-multiple (LCM) of the cycle length of all its guards $\{G_1, \dots, G_n\}$. It is noted $LCM(A)$. For example, the LCM of an automaton with guard cycle lengths of 2,3,4 is 12.

Definition 13. In a time-guarded automaton, $A = \langle Q, \Sigma, \delta, q_0, F, \{G_1, \dots, G_n\} \rangle$ the **length set** of a state $q \in Q$, noted $L_s(q)$ is the set of lengths of all words which can reach the state q . The **modulated length set** of a state q , noted $\lambda(q)$, is the length set after applying $\bmod LCM(A)$ to each member:

$$\lambda(q) = \{a \bmod LCM(A) \mid a \in L_s(q)\}$$

Example 14. For this automaton, A



The length set of q_0 is $\mathbb{N}$ because all word lengths can reach this state (e.g. a^n for all $n \in \mathbb{N}$). The LCM of A is 6, and therefore the modulated length set of q_0 , $\lambda(q_0) = \{0, 1, 2, 3, 4, 5\}$. For q_1 , the length set is also $\mathbb{N}$, because for all $n \in \mathbb{N}$ it is reachable by the word ba^{n-1} . After applying modulu, we get $\lambda(q_1) = \{0, 1, 2, 3, 4, 5\}$.

Definition 15. Transition Odds of Time-Guarded Automata

For a time-guarded automaton $A = \langle Q, \Sigma, \delta, q_0, F, \{G_1, \dots, G_n\} \rangle$, the **odds of a transition** with guard " $|w| = x \bmod y$ " from state u to v , is the fraction of word lengths in $\lambda(u)$ for which the guard's condition holds. The odds of an unguarded transition are equal to 1.

Example 16. In the automaton A from figure 1 above, the odds of the transition with the guard " $0 \bmod 2$ " are:

$$\frac{|\{n \mid n \in \lambda(q_0) \wedge n = 0 \bmod 2\}|}{|\lambda(q_0)|} = \frac{|\{0, 2, 4\}|}{|\{0, 1, 2, 3, 4, 5\}|} = \frac{3}{6} = 0.5$$

For the other transition, with the guard " $0 \bmod 3$ ", the odds are:

$$\frac{|\{0, 3\}|}{|\{0, 1, 2, 3, 4, 5\}|} = \frac{1}{3}$$

Definition 17. Odds of a word over a time-guarded automaton

For a word $w \in \Sigma^*$ and a **deterministic** time-guarded automaton $A = \langle Q, \Sigma, \delta, q_0, F, \{G_1, \dots, G_n\} \rangle$, w 's **odds over** A , $P_A(w)$ are:

- If $w \notin L(A)$, $P_A(w) = 0$

- If $w \in L(A)$, w 's odds over A is the multiplication of all the transition odds, for all the transitions made during A 's run on w .

Example 18. Continuing with our example from figure 1, the odds of the word $aabbb$ are:

$$1 \cdot 1 \cdot \frac{1}{2} \cdot \frac{1}{3} \cdot \frac{1}{2} = \frac{1}{12}$$

Definition 19. p -fluke word over a time-guarded automaton

For a word $w \in \Sigma^*$ and a **deterministic** time-guarded automaton $A = \langle Q, \Sigma, \delta, q_0, F, \{G_1, \dots, G_n\} \rangle$, w is called a p -fluke word over A if $P_A(w) < p$.

Definition 20. For an automaton A , the set of words in $L(A)$ which have length n is noted $L_n(A)$.

Definition 21. For a time-guarded automaton A , the set of words in $L_n(A)$ with minimal odds is called “the lucky set of length n ”, and is noted $S_n(A)$.

$$S_n(A) = \left\{ w \in L_n(A) \mid P_A(w) = \min_{w' \in L_n(A)} P_A(w') \right\}$$

Example 22. The lucky set of length 4 of the automaton A which we viewed in the previous examples is $\{aabb, baab\}$, because both word have odds equal to $\frac{1}{6}$, and all other words in $L_4(A)$ have higher odds.

Proposed algorithm for finding the lucky set of length n of a given time-guarded automaton $A = \langle Q, \Sigma, \delta, q_0, F, \{G_1, \dots, G_n\} \rangle$:

Part 1 - Calculating transition odds.

- (1) Calculate the length of the longest simple cycle in A and note it g . Then note $d = LCM(LCM(A), g)$.
- (2) Initialize an empty hash map H that would map each state $q \in Q$ to $\lambda(q)$.
- (3) Iterate over all words possible words of length d (Σ^d). For each word w , run A on w , trace the states the word passes through, and after each step into a state q' , add the current number of steps to $H[q']$.
- (4) Apply modulu $LCM(A)$ on each member of each set in H , to achieve a hash table of $\lambda(q)$ for each $q \in Q$.
- (5) Iterate over all transitions in δ , and calculate the odds of each one using the λ 's we've calculated.

Part 2 - Finding the lucky set of length n .

- (1) Iterate over all words in Σ^n , run A on each word and calculate the word's odds by multiplying the odds of each transition it passes in its run.
- (2) Find minimal odds among all words, and return the set of words with such odds.

Time-Complexity analysis for each step (Part 1):

- (1) Finding the longest simple cycle takes $O(|Q|!)$ time.
- (2) -
- (3) In the worst case, $d = LCM(A) \cdot |Q|$. We iterate over $|\Sigma|^d$ words, and each run takes d time. There for the worst case time complexity here can be noted as $O(|\Sigma|^d \cdot d) = O(|\Sigma|^{LCM(A) \cdot |Q|} \cdot LCM(A) \cdot |Q|)$.
- (4) $O(|Q| \cdot d) = O(|Q|^2 \cdot LCM(A))$
- (5) $O(|\delta|)$

Worst-case complexity for Part 1: $O(|Q|!) + O\left(|\Sigma|^{LCM(A) \cdot |Q|} \cdot LCM(A) \cdot |Q|\right) + O\left(|Q|^2 \cdot LCM(A)\right) + O(|\delta|)$

Part 2:

- (1) $O(n \cdot |\Sigma|^n)$
- (2) $O(n)$

Total worst-case complexity for Part 2: $O(n \cdot |\Sigma|^n)$.

In conclusion, the worst-case time-complexity of this algorithm is:

$$O(|Q|!) + O\left(|\Sigma|^{LCM(A) \cdot |Q|} \cdot LCM(A) \cdot |Q|\right) + O\left(|Q|^2 \cdot LCM(A)\right) + O(|\delta|) + O(n \cdot |\Sigma|^n)$$

Future research can study the complexity class of this problem.